

Relish Swiggy Instamart Integration Blueprint

A Technical Deep-Dive into AI-Driven Instant Grocery Cart Resolution & Integration Use Cases

1. Executive Summary

Modern consumer cooking workflows suffer from friction between consumption (viewing video recipes) and procurement (acquiring ingredients). **Relish** eliminates this barrier. By combining real-time multimodal dialogue with instant product mappings on **Swiggy Instamart**, home chefs can go from viewing a recipe to receiving fresh, portioned ingredients in under 10 minutes in urban India.

This integration blueprint outlines how the Relish mobile app plans to leverage Model Context Protocol (MCP) tool integrations to connect with Swiggy Instamart's catalog. The agentic workflow focuses on resolving ingredient checklists, planning meal-based stocking, optimizing pack sizes, and executing quick shopping cart checkout handoffs directly to the native Swiggy consumer app.

2. Technical Stack & Architecture

The application utilizes a distributed agentic stack composed of three core components:

- **React Native / Expo Client:** A cross-platform mobile app structured using Expo SDK. Styling is executed with a custom state-driven NativeWind stylesheet. All micro-interactions use local states to manage hover and press transformations. The app captures user audio using PCM streams and plays low-latency voice responses from the server.
- **FastAPI Application Server:** Built with Python to serve as the orchestrator. The server verifies tokens using the Firebase Admin SDK, manages user session databases, and relays full-duplex WebSocket connections between the client and the Gemini Multimodal Live API.
- **Gemini Multimodal Live Engine:** Powered by the **gemini-3.1-flash-live** model running over the *v1alpha Multimodal Live API* protocol. The model receives raw PCM audio, processes user queries under 200ms latency, and returns native speech streams alongside standard tool calls.

2.1. Gemini Live API Integration Details

The 'Kitchen Mode' utilizes a real-time speech relay. When the user says 'Next step' or 'How much sugar did I need?', the client records and chunks microphone audio. It transmits raw 16kHz PCM audio buffers through a WebSocket connection to FastAPI. FastAPI handles authentication and pipes the stream directly into the Gemini Multimodal Live API via the *google-genai* SDK using the **gemini-3.1-flash-live** model. The model responds back with PCM chunks, which the client receives and queues directly to the device sound card.

3. Swiggy Instamart Context & Integration Core

To enable instant ingredient ordering, Relish plans to expose the Swiggy Instamart inventory to the Gemini LLM agent via the Model Context Protocol (MCP). Rather than relying on a static interface, the integration is designed around three primary feature use-cases that directly address cooking friction:

Important Operational Boundary: To prevent accidental purchases and maintain user control, the integration does not allow voice-based ordering, checking out, or payment transactions via the Gemini 3.1 Live Audio connection. Voice interaction is strictly restricted to hands-free kitchen voice assistance, step guidance, and pantry level checking. Users must explicitly initiate cart generation and Swiggy handoff by clicking the physical 'Buy from Instamart' button in the Relish app UI. This compiles the resolved item list and triggers a deep link checkout handoff to the native Swiggy consumer app, where address selection and payment authorization must be completed manually by the user.

3.1. Integration Use Cases

The proposed Swiggy Instamart integration focuses on solving procurement problems dynamically via the AI agent:

Use Case Feature	Agent Role	User Benefit
Order Missing Ingredients	Cross-references the recipe checklist with user's pantry; filters out owned items.	Saves money by only ordering what is actually missing.
Meal-Plan Pantry Restocking	Consolidates ingredient lists across planned meals and schedules a batch restock.	Automates weekly grocery shopping in one tap based on actual plans.
AI-Driven SKU Resolution	Translates recipe descriptions to local dark store SKUs, matching appropriate pack sizes.	Eliminates waste (e.g. buying a 100g butter block instead of 1kg).

3.2. Product Mapping & Conversion Engine

A major challenge is translating raw recipe ingredient lists into concrete purchasable SKUs. A recipe might list *'3 small onions, finely chopped'* or *'a splash of red wine vinegar'*. Swiggy Instamart sells onion in '500g bags' and vinegar in '200ml bottles'. The Product Mapping Engine operates as follows:

- 1. Extraction & Quantifying:** The model parses recipes to extract nominal names, weights, and volumes.
- 2. Catalog Fetching:** The model searches active inventory in local dark stores.
- 3. Semantic Matching & Sizing:** The AI executes an optimization algorithm matching requirements to packaged SKUs. If a recipe calls for 200g of flour, the system selects a 500g package of standard flour rather than a 5kg bag, balancing cost and waste.

4. **Brand Preferences:** The system honors user profile flags (e.g. Organic-only, lowest-price, brand-loyal) when sorting candidate SKUs.

3.3. Cart Deep Linking & checkout Handoff Flow

To protect transaction integrity, payment operations are fully delegated to the native Swiggy consumer application. The checkout workflow is managed as follows:

- **Cart Creation:** Once items are resolved, the backend calls Instamart partner endpoints with the item SKU list, user geolocation, and developer client token. Instamart compiles the cart and returns a unique checkout session token.
- **Deep Linking:** The Relish app opens the Swiggy checkout page directly via a deep link format. The Swiggy app launches with the pre-filled cart ready for the user to confirm address and authorize payment.

4. Reliability & Edge Case Handling Protocols

To guarantee high reliability during operational anomalies, the integration implements the following backup protocols:

- **Catalog Stockout Recovery:** If a specific SKU is out of stock (e.g. particular brand of organic egg), the agent triggers a sub-search query. The agent selects the closest fallback product (e.g., standard free-range eggs) and informs the user via the audio stream: *'I couldn't find organic eggs, so I substituted them with standard free-range eggs. Tap to change.'*
- **Real-time Price Safeguards:** Catalog prices change dynamically. The checkout page checks prices at the final millisecond. If prices diverge by more than 10% from the resolved cost during cart mapping, the client prompts a warning modal.
- **Network Interruptions:** In case of WebSocket drop-offs during voice instructions, the client maintains state snapshots locally. Once reconnected, the agent resumes step-by-step guidance without context loss.